

Digital Radio Lab

Lab Record: 2-bit Full Adder with 7-Segment Display and Arithmetic Logic Unit

Name	Harsh Kumar
Roll No.	226EC5003
Subject	Digital Radio Lab
Experiment	2-bit Full Adder (7-Segment) and ALU
Tool Used	EDA Playground (Verilog HDL)

1. Aim

- To design and implement a 2-bit full adder using structural modelling in Verilog and display the output on a 7-segment display.
- To design and implement an Arithmetic Logic Unit (ALU).

2. Software Used

- EDA Playground (online Verilog/SystemVerilog simulator)
- Simulator: Icarus Verilog 12.0 (or any simulator available on EDA Playground)
- Waveform viewer: EPWave
- Language: Verilog HDL

3. Theory

3.1 Full Adder

A full adder adds three 1-bit inputs (A, B and carry-in Cin) and produces a sum bit and a carry-out bit. $\text{Sum} = A \oplus B \oplus \text{Cin}$ and $\text{Cout} = A.B + \text{Cin}.(A \oplus B)$. It can be built from two XOR gates, two AND gates and one OR gate.

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

3.2 2-bit Full Adder (Ripple Carry, Structural Model)

Two 1-bit full adders are connected in series. The carry-out of the first stage (bits A0, B0, Cin) becomes the carry-in of the second stage (bits A1, B1). The result {Cout, Sum1, Sum0} is a 3-bit number between 0 and 7 (maximum $3 + 3 + 1 = 7$). In **structural modelling** the circuit is described by instantiating gates and sub-modules and connecting them with wires, just like drawing a schematic in text form.

3.3 Seven-Segment Display

A 7-segment display has seven LEDs (a to g) arranged to form digits. In a **common cathode** display, a segment glows when its input is logic 1. A decoder converts the 4-bit binary value into the 7 segment signals. Here the 3-bit adder output {Cout, Sum} is padded with a 0 to form a 4-bit value and passed to the decoder. The segment code is written in the order **abcdefg**.

Digit	a	b	c	d	e	f	g	Digit	a	b	c	d	e	f	g
0	1	1	1	1	1	1	0	4	0	1	1	0	0	1	1
1	0	1	1	0	0	0	0	5	1	0	1	1	0	1	1
2	1	1	0	1	1	0	1	6	1	0	1	1	1	1	1
3	1	1	1	1	0	0	1	7	1	1	1	0	0	0	0

3.4 Arithmetic Logic Unit (ALU)

An ALU is the combinational block of a processor that performs arithmetic and logical operations on its operands. The operation is chosen by select lines. The 4-bit ALU designed here has two 4-bit inputs A and B, a 3-bit select input SEL, a 4-bit output Y, a Carry/Borrow flag and a Zero flag.

SEL	Operation	Function
000	ADD	$Y = A + B$
001	SUBTRACT	$Y = A - B$
010	AND	$Y = A \& B$
011	OR	$Y = A B$
100	XOR	$Y = A \wedge B$
101	NOT	$Y = \sim A$
110	INCREMENT	$Y = A + 1$
111	DECREMENT	$Y = A - 1$

The Carry flag is 1 when an addition/increment produces a carry-out, or when a subtraction/decrement needs a borrow ($A < B$). The Zero flag is 1 when the result Y is 0000.

4. Source Code (Verilog)

In EDA Playground, the **design** module goes in **design.sv** and the **testbench** goes in **testbench.sv**. Each experiment below is therefore given as two separate listings.

4.1 2-bit Full Adder (Structural) with 7-Segment Display

design.sv (Design window - right side in EDA Playground)

```
1  // ===== DESIGN: 1-bit Full Adder (structural, gate level) =====
2  module full_adder (
3      input  a, b, cin,
4      output sum, cout
5  );
6      wire x1, a1, a2;
7
8      xor g1 (x1,  a,  b);      // x1 = a ^ b
9      xor g2 (sum, x1, cin);    // sum = a ^ b ^ cin
10     and g3 (a1,  a,  b);      // a1 = a & b
11     and g4 (a2,  x1, cin);    // a2 = (a ^ b) & cin
12     or  g5 (cout, a1, a2);    // cout = a1 | a2
13 endmodule
14
15 // ===== DESIGN: 2-bit Full Adder (structural, ripple carry) =====
16 module adder2bit (
17     input  [1:0] a, b,
18     input      cin,
19     output [1:0] sum,
20     output      cout
21 );
22     wire c1;                  // carry between the two stages
23
24     full_adder fa0 (.a(a[0]), .b(b[0]), .cin(cin), .sum(sum[0]), .cout(c1));
25     full_adder fa1 (.a(a[1]), .b(b[1]), .cin(c1), .sum(sum[1]), .cout(cout));
26 endmodule
27
28 // ===== DESIGN: BCD / Hex to 7-segment decoder (common cathode) =====
29 // seg = {a, b, c, d, e, f, g} (1 = segment ON)
30 module seg7_decoder (
31     input  [3:0] d,
32     output reg [6:0] seg
33 );
34     always @(*) begin
35         case (d)
36             4'h0 : seg = 7'b1111110;
37             4'h1 : seg = 7'b0110000;
38             4'h2 : seg = 7'b1101101;
39             4'h3 : seg = 7'b1111001;
40             4'h4 : seg = 7'b0110011;
41             4'h5 : seg = 7'b1011011;
42             4'h6 : seg = 7'b1011111;
43             4'h7 : seg = 7'b1110000;
44             4'h8 : seg = 7'b1111111;
45             4'h9 : seg = 7'b1111011;
46             4'hA : seg = 7'b1110111;
47             4'hB : seg = 7'b0011111;
48             4'hC : seg = 7'b1001110;
49             4'hD : seg = 7'b0111101;
50             4'hE : seg = 7'b1001111;
51             4'hF : seg = 7'b1000111;
52             default : seg = 7'b0000000;
53         endcase
54     end
55 endmodule
56
57 // ===== DESIGN: Top module (adder + 7-segment display) =====
58 module adder_7seg (
```

```

59 input [1:0] a, b,
60 input      cin,
61 output [1:0] sum,
62 output      cout,
63 output [6:0] seg
64 );
65 adder2bit add (.a(a), .b(b), .cin(cin), .sum(sum), .cout(cout));
66
67 // {cout, sum} is the 3-bit result (0 to 7); pad with 0 to make 4 bits
68 seg7_decoder dec (.d({1'b0, cout, sum}), .seg(seg));
69 endmodule

```

testbench.sv (Testbench window - left side in EDA Playground)

```

1  // ===== TESTBENCH =====
2  module adder_7seg_tb;
3      reg [1:0] a, b;
4      reg      cin;
5      wire [1:0] sum;
6      wire      cout;
7      wire [6:0] seg;
8
9      adder_7seg dut (a, b, cin, sum, cout, seg);
10
11     integer k;
12
13     initial begin
14         $dumpfile("dump.vcd");
15         $dumpvars(0, adder_7seg_tb);
16
17         $display("A B Cin | Cout Sum | Dec | Seg(abcdefg)");
18         for (k = 0; k < 32; k = k + 1) begin
19             {a, b, cin} = k[4:0];
20             #10;
21             $display("%b %b %b | %b %b | %0d | %b",
22                     a, b, cin, cout, sum, {cout, sum}, seg);
23         end
24         $finish;
25     end
26 endmodule

```

4.2 4-bit Arithmetic Logic Unit

design.sv (Design window - right side in EDA Playground)

```

1  // ===== DESIGN: 4-bit Arithmetic Logic Unit =====
2  module alu4bit (
3      input [3:0] a, b,
4      input [2:0] sel,
5      output reg [3:0] y,
6      output reg      carry, // carry-out (ADD/INC) or borrow (SUB/DEC)
7      output          zero  // 1 when the result is 0
8  );
9      reg [4:0] t; // 5-bit temporary to capture carry/borrow
10
11     always @(*) begin
12         t = 5'b00000;
13         case (sel)
14             3'b000 : t = a + b; // ADD
15             3'b001 : t = a - b; // SUBTRACT
16             3'b010 : t = {1'b0, a & b}; // AND
17             3'b011 : t = {1'b0, a | b}; // OR
18             3'b100 : t = {1'b0, a ^ b}; // XOR
19             3'b101 : t = {1'b0, ~a}; // NOT A
20             3'b110 : t = a + 1; // INCREMENT A
21             3'b111 : t = a - 1; // DECREMENT A
22         endcase

```

```

23     y      = t[3:0];
24     carry = t[4];
25 end
26
27     assign zero = (y == 4'b0000);
28 endmodule

```

testbench.sv (Testbench window - left side in EDA Playground)

```

1  // ===== TESTBENCH =====
2  module alu4bit_tb;
3      reg [3:0] a, b;
4      reg [2:0] sel;
5      wire [3:0] y;
6      wire      carry, zero;
7
8      alu4bit dut (a, b, sel, y, carry, zero);
9
10     integer k;
11
12     task run_all;
13     begin
14         for (k = 0; k < 8; k = k + 1) begin
15             sel = k;
16             #10;
17             $display("%b | %b %b | %b | %b | %b", sel, a, b, y, carry, zero);
18         end
19     end
20 endtask
21
22 initial begin
23     $dumpfile("dump.vcd");
24     $dumpvars(0, alu4bit_tb);
25
26     $display("SEL | A    B    | Y    | Carry | Zero");
27
28     a = 4'b1010; b = 4'b0011; run_all; // A = 10, B = 3
29     $display("-----");
30     a = 4'b0110; b = 4'b1001; run_all; // A = 6, B = 9
31     $display("-----");
32     a = 4'b1111; b = 4'b0001; run_all; // A = 15, B = 1
33     $finish;
34 end
35 endmodule

```

5. Procedure

1. Open EDA Playground (edaplayground.com) and log in.
2. Select **Testbench + Design** language as **SystemVerilog/Verilog** and choose the simulator (Icarus Verilog).
3. Type the design modules (full_adder, adder2bit, seg7_decoder, adder_7seg) in the **design.sv** window and the testbench in the **testbench.sv** window (the testbench is not to be mixed with the design code).
4. Tick **Open EPWave after run** and click **Run**.
5. Observe the output table printed in the console and verify the sum, carry and 7-segment pattern for all 32 input combinations.
6. Open EPWave and observe the waveforms of a, b, cin, sum, cout and seg.
7. Clear both windows, type the ALU design and its testbench, and run again.
8. Check the result Y, Carry and Zero flags for each SEL value (000 to 111) against the expected values.
9. Take screenshots of the console output and waveforms and attach them to the record.

6. Observations

6.1 2-bit Full Adder with 7-Segment Display

A1A0	B1B0	Cin	Cout	Sum1Sum0	Decimal	7-seg (abcdefg)
00	00	0	0	00	0	1111110
00	00	1	0	01	1	0110000
00	01	0	0	01	1	0110000
00	01	1	0	10	2	1101101
00	10	0	0	10	2	1101101
00	10	1	0	11	3	1111001
00	11	0	0	11	3	1111001
00	11	1	1	00	4	0110011
01	00	0	0	01	1	0110000
01	00	1	0	10	2	1101101
01	01	0	0	10	2	1101101
01	01	1	0	11	3	1111001
01	10	0	0	11	3	1111001
01	10	1	1	00	4	0110011
01	11	0	1	00	4	0110011
01	11	1	1	01	5	1011011
10	00	0	0	10	2	1101101
10	00	1	0	11	3	1111001
10	01	0	0	11	3	1111001
10	01	1	1	00	4	0110011
10	10	0	1	00	4	0110011
10	10	1	1	01	5	1011011
10	11	0	1	01	5	1011011
10	11	1	1	10	6	1011111
11	00	0	0	11	3	1111001

A1A0	B1B0	Cin	Cout	Sum1Sum0	Decimal	7-seg (abcdefg)
11	00	1	1	00	4	0110011
11	01	0	1	00	4	0110011
11	01	1	1	01	5	1011011
11	10	0	1	01	5	1011011
11	10	1	1	10	6	1011111
11	11	0	1	10	6	1011111
11	11	1	1	11	7	1110000

6.2 4-bit ALU

Case 1: A = 1010 (10), B = 0011 (3)

SEL	Operation	Y	Y (decimal)	Carry/Borrow	Zero
000	ADD	1101	13	0	0
001	SUB	0111	7	0	0
010	AND	0010	2	0	0
011	OR	1011	11	0	0
100	XOR	1001	9	0	0
101	NOT A	0101	5	0	0
110	INC A	1011	11	0	0
111	DEC A	1001	9	0	0

Case 2: A = 0110 (6), B = 1001 (9)

SEL	Operation	Y	Y (decimal)	Carry/Borrow	Zero
000	ADD	1111	15	0	0
001	SUB	1101	13	1	0
010	AND	0000	0	0	1
011	OR	1111	15	0	0
100	XOR	1111	15	0	0
101	NOT A	1001	9	0	0
110	INC A	0111	7	0	0
111	DEC A	0101	5	0	0

Case 3: A = 1111 (15), B = 0001 (1)

SEL	Operation	Y	Y (decimal)	Carry/Borrow	Zero
000	ADD	0000	0	1	1
001	SUB	1110	14	0	0
010	AND	0001	1	0	0
011	OR	1111	15	0	0
100	XOR	1110	14	0	0
101	NOT A	0000	0	0	1
110	INC A	0000	0	1	1
111	DEC A	1110	14	0	0

6.3 Simulation Output (Screenshots)

Experiment 1: 2-bit Full Adder with 7-Segment Display

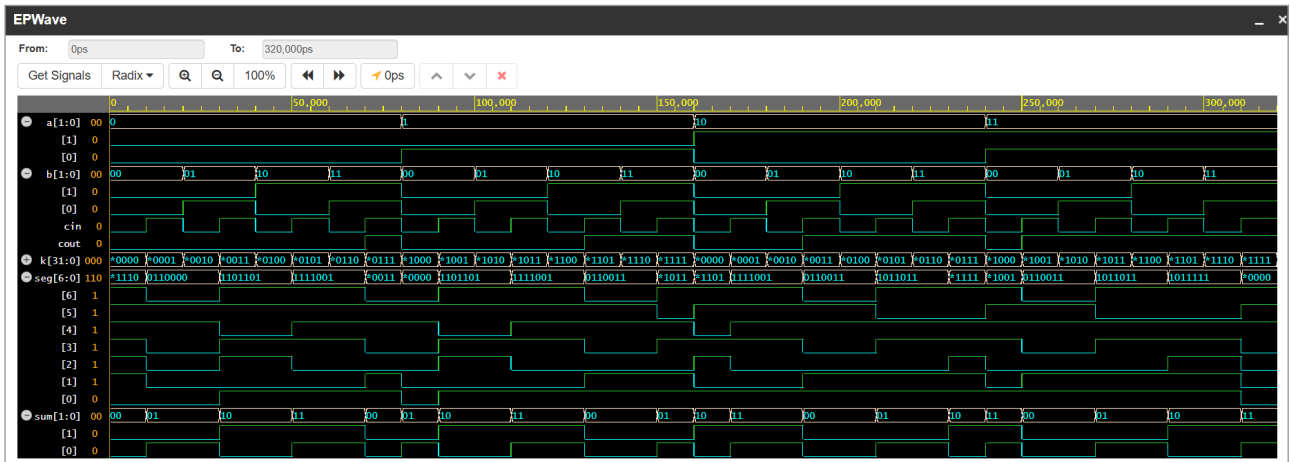


Fig. 1: EPWave output of the 2-bit full adder with 7-segment decoder (a, b, cin, k, seg, sum, cout)

Experiment 2: 4-bit Arithmetic Logic Unit

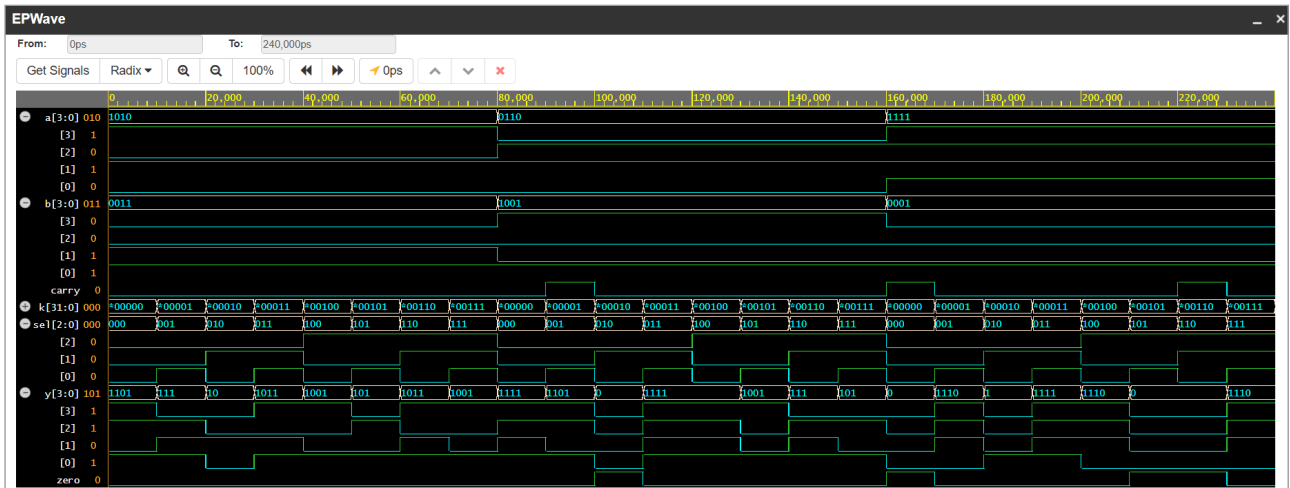


Fig. 2: EPWave output of the 4-bit ALU (a, b, carry, k, sel, y, zero)

7. Results & Discussion

- The 2-bit structural full adder produced the correct sum and carry-out for all 32 combinations of A, B and Cin. The largest result ($3 + 3 + 1 = 7$) was obtained with Cout = 1 and Sum = 11.
- The 3-bit result {Cout, Sum} was correctly decoded by the 7-segment decoder, and the segment patterns for digits 0 to 7 matched the common cathode truth table.
- The ALU performed all eight operations correctly. For example, with A = 10 and B = 3, ADD gave 13 (1101) and SUB gave 7 (0111).
- The Carry/Borrow flag was set correctly: it went high for $6 - 9$ (borrow) and for $15 + 1$ (ADD) and for the increment of 15 (carry-out), where the result wrapped around to 0000.
- The Zero flag was set whenever the result was 0000, such as in $15 + 1$ and in $6 \text{ AND } 9$.
- Both designs are combinational, so the outputs changed as soon as the inputs changed.

8. Conclusion

A 2-bit full adder was designed using structural modelling with gate-level full adders connected in ripple-carry form, and its output was displayed through a 7-segment decoder. A 4-bit ALU with eight arithmetic and logical operations and carry and zero flags was also designed. Both circuits were simulated in EDA Playground and the results matched the expected values, hence the experiment was successfully completed.